

CityBikes

pipeline de datos end-to-end

Bronze → Silver → Gold · Airflow · PostgreSQL · Streamlit · Docker

INTEGRANTES · GRUPO G04

Di Lacio, Lautaro

@DiLacio-Lautaro

Melograna, Federico

@Melograma-Federico

Rial, Alejo

@Alejo-Rial

Lust, Tobias

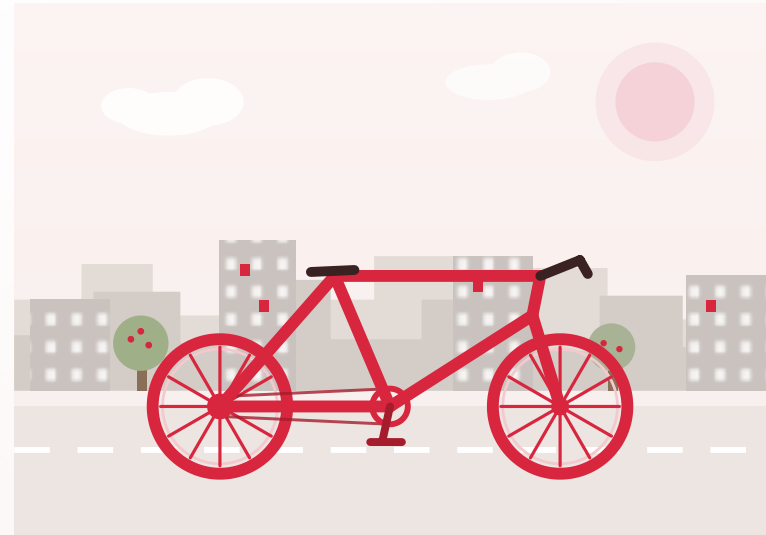
@lust-tobias

**Quinteros Amicone,
Lautaro**

@Quinteros-Lautaro

Romero, Manuel

@romero-rodrigo



Datos de bici pública,
en vivo, de tres ciudades.

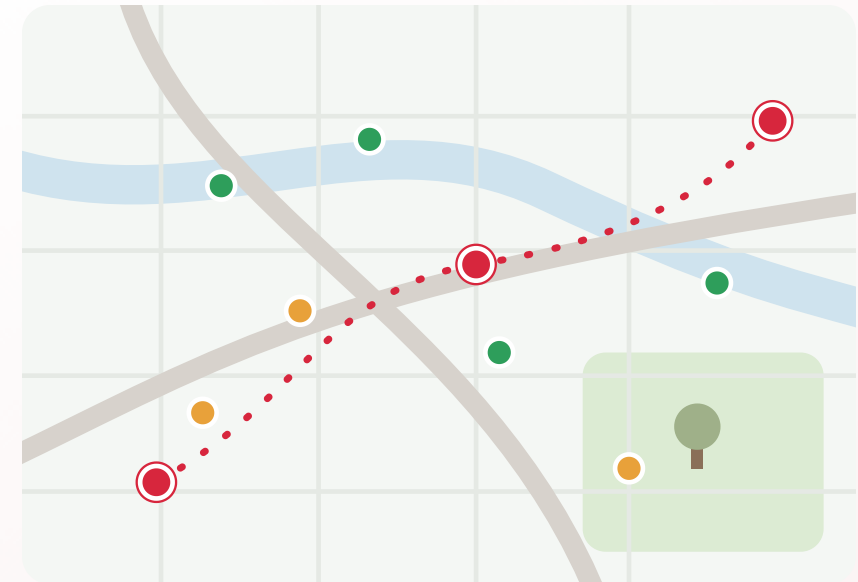
La API y la pregunta de negocio

LA PREGUNTA DE NEGOCIO

¿Qué estaciones **se saturan** o se quedan **sin bicis**, y a qué horas del día?

Fuente: CityBikes API

- `api.citybik.es/v2` — sin autenticación.
- Estado en vivo: bicis libres, slots vacíos y ubicación de cada estación.
- **Refresca cada 2–5 min** → justifica Airflow acumulando snapshots.



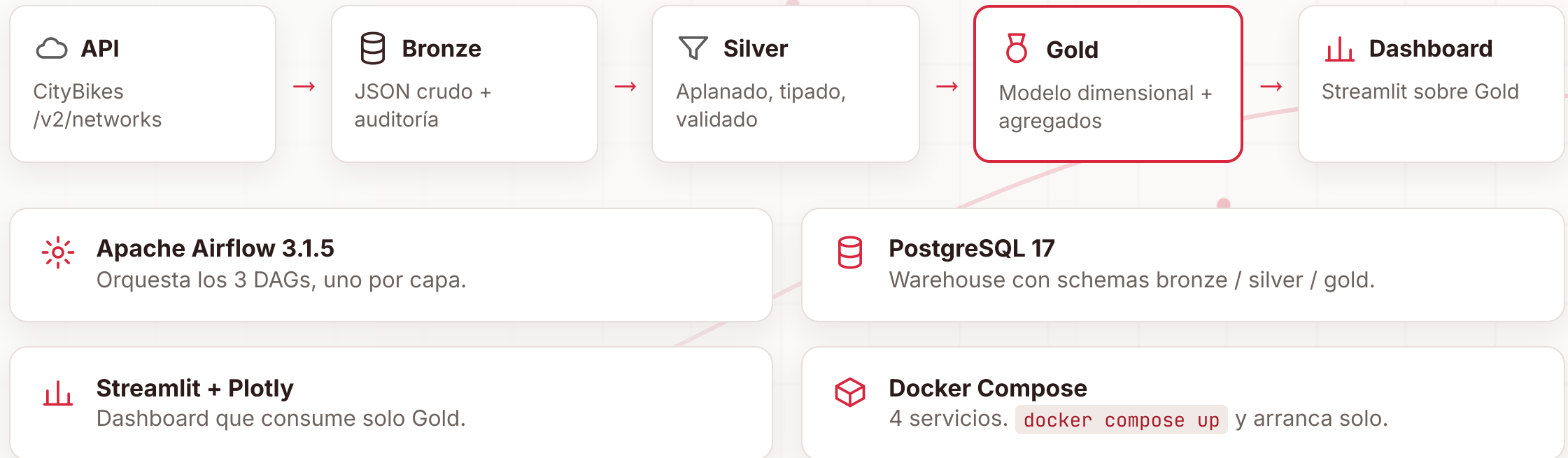
● Equilibrada ● Llenándose ● Saturada / vacía

EcoBici Cd de México

Bicing Barcelona

Divvy Chicago

Arquitectura Medallion + Stack



Qué hace cada capa

**Bronze**
`citybikes_stations_raw` — una fila por estación × snapshot con el JSON crudo (`JSONB`) + `ingested_at` . Sin tocar.

**Silver**
`station_status` — aplanada, tipada, validada y descartada inactivas. Calcula `occupancy_rate` y flags `is_empty` / `is_full` . Incremental por watermark.

**Gold**
`dim_network` , `dim_station` , `fact_station_hourly` (estación×hora) y `station_current` (foto en vivo).

DAG	Schedule	Arranca solo
<code>citybikes_bronze</code>	cada 5 min	Sí · <code>is_paused_upon_creation=False</code>
<code>citybikes_silver</code>	cada 10 min	Sí · idempotente (ON CONFLICT)
<code>citybikes_gold</code>	cada 15 min	Sí · recalcula últimas 3 h

Los tres DAGs arrancan activos al hacer `docker compose up` — sin tocar nada a mano.

El dashboard (Gold)

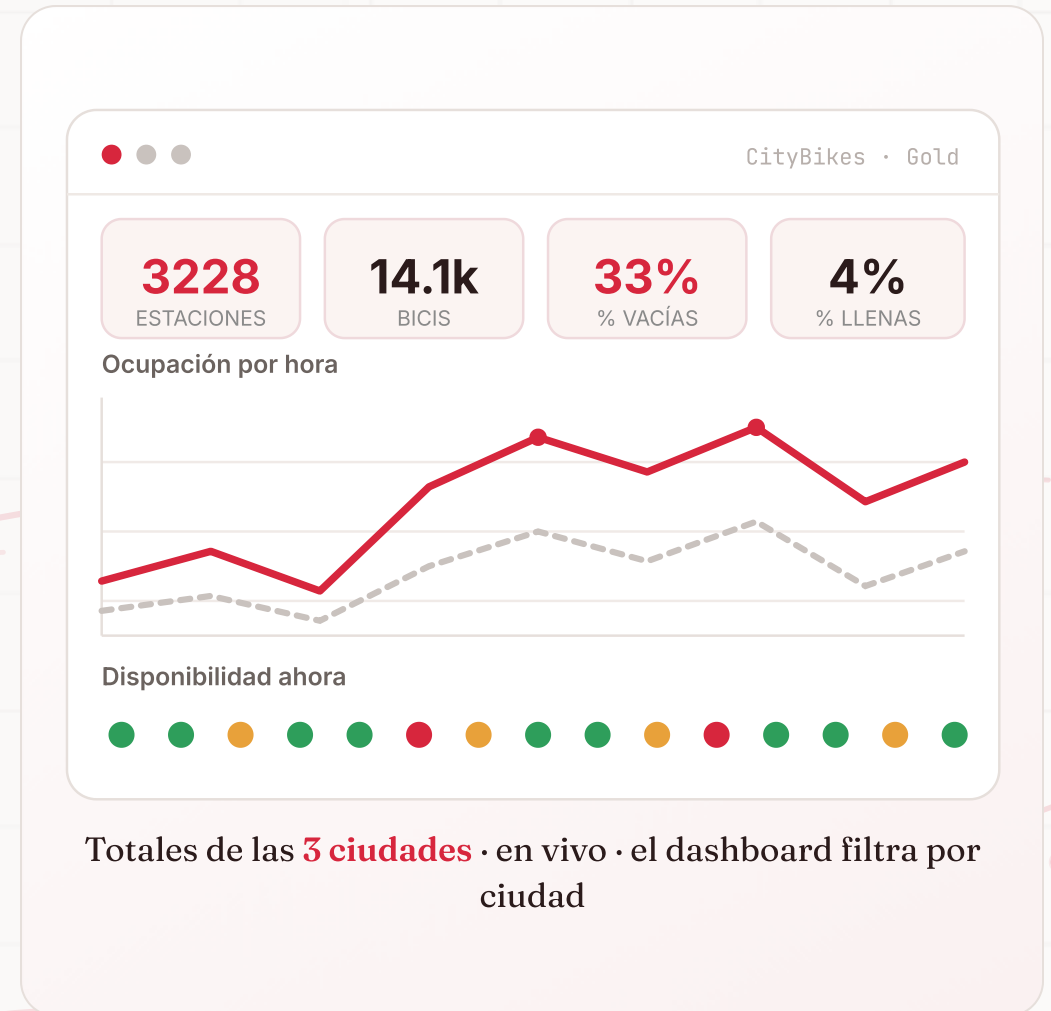
Consume **exclusivamente** las tablas Gold — el modelo final de negocio.

- **KPIs en vivo:** estaciones, bicis disponibles, % vacías, % llenas.
- **Mapa de disponibilidad:** color por ocupación, tamaño por capacidad.
- **Patrón por hora del día:** ocupación y % de estaciones vacías.
- **Estaciones críticas:** top 10 sin bicis y top 10 saturadas.
- **Comparación entre ciudades** y filtro por red que afecta todo.

🕒 Demo en vivo → localhost:8501

LO QUE ENCONTRAMOS

Chicago es la más crítica (~43% sin bicis); **Barcelona** la más equilibrada (~12%). El problema dominante: la **falta de bicis**.



Dificultades y arranque automático

⚠ Dificultades y cómo las resolvimos

- `timestamp` de la API inconsistente → usamos nuestro `ingested_at` como eje temporal.
- `empty_slots` a veces es `null` → se normaliza en Silver.
- ~560 redes en la API → fijamos 3 para comparar y quedar bajo el rate limit (~36 de 300 req/h).
- Re-correr DAGs sin duplicar → watermark + `ON CONFLICT`.

✓ Arranca solo

- `docker compose up` levanta los 4 servicios.
- Schemas creados vía `init.sql` montado en Postgres.
- DAGs activos por default, con schedule definido.
- Datos en minutos: Bronze ~5', Silver ~10', Gold ~15'.

¿Preguntas? · ¡Gracias por ver!

